

Chapter Four

System Implementation and Results

4.1 Dataset

The proposed AI Violence Detection System depends on two related dataset groups. The first group contains CCTV and real-world video clips for violence, fighting, and abnormal behavior recognition. The second group contains annotated weapon images for firearms, knives, sticks, and rods. Combining both groups helps the system detect the action itself and also recognize dangerous objects that may increase the alert level.

Dataset Sources Used in the Project

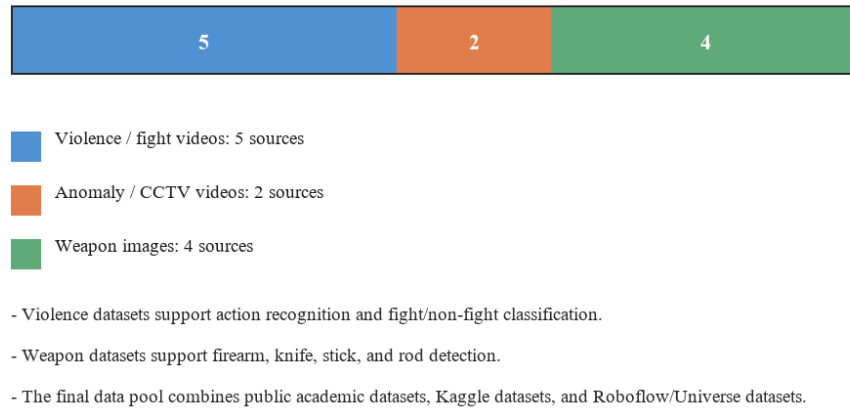


Figure 4.1: Dataset source groups used in the project

The datasets used in this project are listed in Table 4.1. The video datasets were used to support the violence detection model, while the object detection datasets were used to train and validate weapon detection behavior.

No.	Dataset / Source	Main Use
1	Mohamed et al. (2019), Violence Recognition from Videos using Deep Learning Techniques	Violence and non-violence video recognition
2	Atharva et al. (2025), Real-Time CCTV Violence Detection Automation System	Real-time CCTV violence examples
3	Aremu et al. (2024), Smart-City CCTV Violence Detection Dataset (SCVD)	Smart-city surveillance violence detection
4	Fransco Perez Hernandez et al. (2020), OD-WeaponDetection: Knife Classification	Knife object detection/classification
5	Ali Alrabeei et al. (2024), CCTV Guns Online	Gun detection in CCTV scenes
6	Violence Detection et al. (2024),	Stick and rod object detection

	Stick/Rod Detection	
7	S Kumar et al. (2025), Firearms Detection	Firearm detection
8	Dania Zehra et al. (2025), RWF-2000	Fight/non-fight video classification
9	Shrey Joshi et al. (2021), NTU CCTV-Fights Dataset	CCTV fight detection
10	Avdhesh Chaudhary et al. (2023), UCF Crime Dataset	CCTV anomaly and violent crime scenes

Table 4.1: Datasets used in the project

4.2 Description of Software Tools

Several tools were used to implement the web application, the artificial intelligence services, and the live monitoring dashboard. The most important AI and computer vision tools are shown in Table 4.2.

Tool	Version	Use in the Project
Flask	3.1.3	Builds the lightweight Python APIs for weapon and violence detection streams.
TensorFlow	2.21.0	Loads and runs the trained violence detection model.
Keras	3.14.0	Provides the model interface used for the violence classification network.
OpenCV	4.13.0.92	Captures frames, resizes images, draws overlays, saves snapshots, and records clips.
Laravel	12.x	Provides authentication, database models, dashboard pages, and incident API endpoints.
React / Vite	Project frontend stack	Builds the operator dashboard, stream controls, and incident table.
MySQL	Project database	Stores cameras, incidents, confidence scores, alert levels, and evidence links.

Table 4.2: Main software tools used in implementation

4.3 Experimental Setup and Results

The implemented system is divided into a Laravel web backend, a React dashboard, and two Python AI services. The weapon service runs on port 5000 and the violence service runs on port 5001. Each service provides a live MJPEG stream and start/stop endpoints for controlling analysis from the dashboard.

AI Violence Detection System Architecture

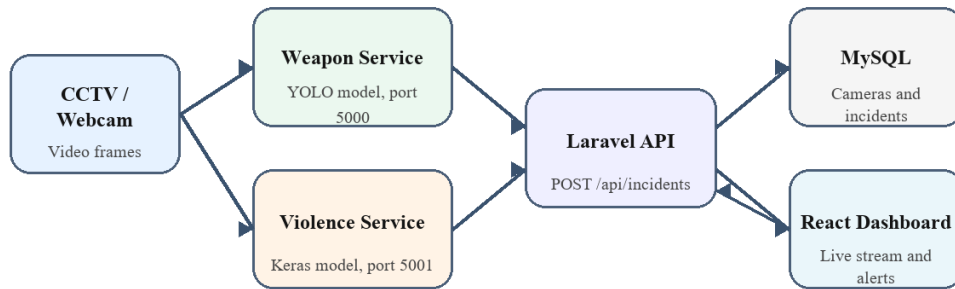


Figure 4.2: System implementation architecture

4.3.1 Data Preprocessing

For violence detection, frames are captured from the camera or video source, resized, converted to floating point values, and passed through the MobileNetV2 preprocessing function. The system collects a short sequence of 20 frames before prediction because violence is an action that depends on motion over time. For weapon detection, frames are resized and passed to the YOLO model, which returns bounding boxes and confidence scores.

Preprocessing Pipeline

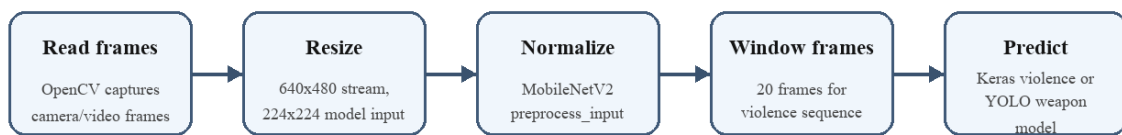


Figure 4.3: Data preprocessing pipeline

- Frame size used in the live stream: 640 x 480 pixels.
- Violence model input size: 224 x 224 pixels.
- Violence decision threshold: 0.60.

- Weapon confidence threshold: 0.65.

4.3.2 Model Training

The violence model follows a deep learning sequence-classification approach. MobileNetV2 is used as a feature extractor for individual frames, and the extracted features are passed to a trained Keras model saved as `cctvmodel_advanced.keras`. This design reduces the cost of learning visual features from scratch and allows the classifier to focus on the final violence/non-violence decision.

The weapon model uses a YOLO object detection model saved as `weapon_detection_yolov11m.pt`. YOLO is suitable for real-time detection because it predicts object locations and classes in one pass. In the implementation, the service filters firearm-related class names, tracks detections using ByteTrack, and starts evidence recording only after consecutive hits to reduce false alerts.

4.3.3 Evaluation Metrics

The following metrics are suitable for evaluating the project models:

- Accuracy = correctly classified samples / total samples.
- Precision = true positives / (true positives + false positives).
- Recall = true positives / (true positives + false negatives).
- F1-score = $2 \times (\text{precision} \times \text{recall}) / (\text{precision} + \text{recall})$.
- Mean Average Precision (mAP) is used for weapon object detection because it measures both classification and localization quality.
- Response time is used at system level to check whether the stream and alert creation are fast enough for live monitoring.

4.3.4 Results Presentation

The implemented system produces visible and stored outputs when a suspicious event is detected. The AI service draws detection overlays on the stream, saves a snapshot, records a short video clip, and sends an incident payload to the Laravel API. Laravel stores the event in the database and the dashboard displays it in the incident table.

Incident Output Flow

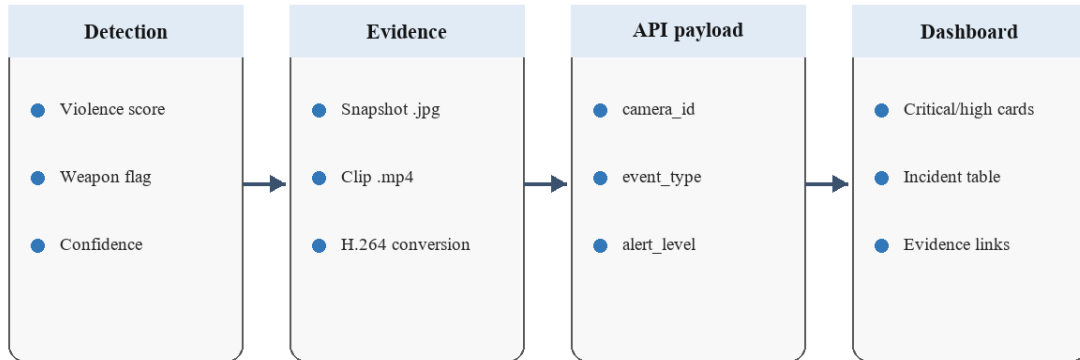


Figure 4.4: Incident output flow

Test Case	Expected Result	Observed Implementation Result
Open weapon stream	MJPEG stream is available on port 5000	Implemented through /stream endpoint.
Open violence stream	MJPEG stream is available on port 5001	Implemented through /stream endpoint.
Start/stop AI	Dashboard can control Python analysis	Implemented through /start and /stop endpoints.
Weapon detection	Critical incident is created when weapon confidence passes threshold	Payload includes weapon_detected=true and alert_level=critical.
Violence detection	High incident is created when violence score passes threshold	Payload includes violence_score and alert_level=high.
Evidence saving	Snapshot and clip are saved for review	Files are saved under storage/app/public/incidents.
Dashboard update	New incidents appear without manual refresh	Dashboard polls latest incidents every five seconds.

Table 4.3: Functional results of the implemented system

4.3.5 Comparative Analysis

Compared with a simple single-model violence classifier, the proposed system adds a second weapon detection service. This improves practical safety monitoring because a scene can be dangerous even when the motion pattern is not clearly classified as violence. Compared with an offline notebook experiment, the implemented system provides a complete application pipeline: live stream, inference service, evidence storage, incident API, database records, and dashboard review.

Aspect	Single Offline Model	Proposed System
Input	Prepared videos or images	Live webcam/CCTV stream
Detection scope	Usually one task	Violence detection and weapon detection
Output	Prediction score	Alert level, snapshot, clip, dashboard record
Deployment	Experiment-only environment	Flask AI services connected to Laravel dashboard
Review	Manual checking	Incident archive with evidence links

Table 4.4: Comparison between offline detection and the proposed implemented system

4.4 Discussion

The implementation shows that deep learning can be connected with a practical surveillance dashboard. The violence model handles action recognition through frame sequences, while the weapon model handles object detection. The dashboard and incident API make the system more useful than a standalone model because the operator can monitor events, inspect evidence, and review alert history.

The main limitation is that real-time performance depends on the camera source, CPU/GPU capability, and model size. Also, lighting, occlusion, crowd density, and camera angle can affect detection confidence. For this reason, the system stores confidence values and evidence files instead of relying only on a binary alert.

Chapter Five

Run the Application

5.1 Running the Web Application

This chapter explains how to run and use the application. The project can be started locally by running the Laravel backend, the Vite frontend, and one of the Python AI services.

Application Running Workflow



Figure 5.1: Main running workflow

1. Install PHP and Node dependencies using composer install and npm install.
2. Create the .env file, configure MySQL, generate the application key, and run database migrations.
3. Start Laravel on <http://127.0.0.1:8000> and start Vite for frontend assets.
4. Run the weapon service or violence service from the ai_service folder.
5. Open the login page and enter the dashboard.

5.2 Running the AI Services

The weapon service is started by running `ai_service/weapon_detection_service.py`. It loads the YOLO model from `storage/app/models/weapon/weapon_detection_yolov11m.pt` and exposes a stream on <http://127.0.0.1:5000/stream>.

The violence service is started by running `ai_service/violence_detection_service.py`. It loads the Keras model from `storage/app/models/violence/cctvmodel_advanced.keras` and exposes a stream on <http://127.0.0.1:5001/stream>.

Service	Local URL	Purpose
Laravel backend	http://127.0.0.1:8000	Dashboard and incident API
Weapon AI	http://127.0.0.1:5000	Weapon detection stream
Violence AI	http://127.0.0.1:5001	Violence detection stream
MySQL	127.0.0.1:3306	Application database

Table 5.1: Main local services

5.3 Dashboard Usage

After logging in, the operator can choose either weapon detection or violence detection, load the stream, and press Start AI. The dashboard shows the active stream, current system status, number of cameras, and recent incidents. When the AI service detects an event, it posts the incident to Laravel and the dashboard displays the alert with confidence and evidence links.

- Start webcam: opens the browser camera preview.
- Load stream: loads the selected Python AI stream into the dashboard.
- Start AI: sends a request to the selected Python service to begin analysis.
- Stop AI: pauses analysis and closes the current recording if needed.
- Incident archive: lists event type, camera, confidence, detected time, snapshot, and clip.

5.4 Evidence Review

Each incident can contain a snapshot and a recorded video clip. These files are saved in the public storage path and linked from the dashboard. This makes the alert review process easier because the operator can inspect what caused the detection instead of depending only on the model score.

Chapter Six

Conclusion and Future Work

6.1 Conclusion

This project implemented an AI-based violence detection and weapon detection system for CCTV-style monitoring. The final system combines Python AI services, OpenCV video processing, TensorFlow/Keras violence classification, YOLO weapon detection, a Laravel backend, a MySQL database, and a React dashboard.

The implementation achieved the main project goal: detecting suspicious events from a live stream, creating incident records, saving evidence, and presenting alerts to the user through a web dashboard. The separation between the Laravel backend and Python AI services also makes the system easier to maintain because model inference and web application logic are handled independently.

6.2 Limitations

- The system performance depends on camera quality, lighting conditions, and available hardware.
- Running two OpenCV services on the same webcam may fail on Windows because one camera is usually locked by one process.
- The current dashboard uses polling every five seconds; real-time WebSocket alerts would be faster.
- Final accuracy values should be reported after running the trained models on a fixed test split and saving the evaluation logs.

6.3 Future Work

- Create a combined AI service that opens the camera once and runs both violence and weapon models on the same frames.
- Add WebSocket notifications for instant dashboard alerts.
- Improve the dataset by adding more local CCTV scenes with different lighting, camera angles, and crowd densities.
- Add role-based incident review actions such as confirmed, false alarm, ignored, and escalated.
- Deploy the system on an edge device or GPU-enabled server for better real-time performance.
- Add model evaluation reports directly to the dashboard, including confusion matrix, precision, recall, F1-score, and mAP.

6.4 References

- Mohamed et al., "Violence Recognition from Videos using Deep Learning Techniques," 2019. <https://ieeexplore.ieee.org/abstract/document/9014714>
- Atharva et al., "Real-Time CCTV Violence Detection Automation System," 2025. <https://drive.google.com/file/d/1dXrt-dXBmjBao3yIBLhqTv4e3iKti7Hw/view>
- Aremu et al., "Smart-City CCTV Violence Detection Dataset (SCVD)," 2024. <https://www.kaggle.com/datasets/toluwaniaremu/smartcity-cctv-violence-detection-dataset-scvd>
- Fransco Perez Hernandez et al., "OD-WeaponDetection: Knife Classification," 2020. <https://datasetninja.com/od-weapon-detection-knife-classification>
- Ali Alrabeei et al., "CCTV Guns Online," 2024. <https://universe.roboflow.com/ali-alrabeei-qblzy/cctv-guns-online>
- Violence Detection et al., "Stick/Rod Detection," 2024. <https://universe.roboflow.com/violence-detection-rvh0k/stick-rod-detection>
- S Kumar et al., "Firearms Detection," 2025. <https://universe.roboflow.com/s-kumar-baywt/firearms-detection-5wjh7>
- Dania Zehra et al., "RWF-2000," 2025. <https://www.kaggle.com/datasets/daniazehra/rwf-2000>
- Shrey Joshi et al., "NTU CCTV-Fights Dataset," 2021. <https://www.kaggle.com/datasets/shreyj1729/cctv-fights-dataset/data>
- Avdhesh Chaudhary et al., "UCF Crime Dataset," 2023. <https://www.kaggle.com/datasets/webadvisor/real-time-anomaly-detection-in-cctv-surveillance>